QALight

Frontend-розробка

Заняття 15. Об'єкти

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Об'єкти
Структури даних. Властивості об'єкта. Методи об'єкту. Контекст this та лексичне середовище.

Створення об’єкта calculator, що має механізм ланцюжкових викликів

Об'єкти

Структури даних. Властивості об'єкта. Методи об'єкту. Контекст this та лексичне середовище.

Об’єкт JavaScript

Теорія: Об’єкт JavaScript

Об’єкт javascript – це складний тип даних, що об’єднує множину властивостей у загальний модуль і дозволяє зберігати та отримувати за допомогою запиту значення по їх іменам.
Майже всі **об’єкти в JavaScript є екземплярами Object**; типовий об'єкт успадковує властивості (включно з методами) від **Object.prototype**, хоча ці властивості можуть бути перевизначені. Єдині об’єкти, які не успадковують від Object.prototype, це об’єкти з нульовим прототипом або ті, що походять від інших об’єктів нульового прототипу.

Об’єкт має пари ключ-значення, що дозволяє отримувати значення за допомогою пов’язаних ключів.

```
const address = {
      street: '13/25 New Avenue',
      city: 'New Heaven ',
      country: 'Ukraine'
  },
```

Кожна пара ключ/значення має бути розділена комою, а ключ та значення в кожному випадку відокремлені двокрапкою.

Властивість об’єкта

Теорія: Об’єкт JavaScript

Об’єкт можна створити за допомогою фігурних дужок {…} з необов’язковим списком властивостей. Кожна властивість представлена **іменем** і **значенням**, що асоційовано з цим іменем. **Властивість** – це пара “**ключ: значення**”, де ключ – це рядок (також називається “ім’я властивості”), а значення може бути будь-яким.

Значенням властивості може бути функція, яку називають **методом** об’єкта.

Властивість об’єкта - це змінна, що закріплена за об’єктом, і є звичайною змінною JavaScript. Отримати доступ до властивості об’єкта можна за допомогою дот-нотації: **objectName.propertyName**

Ім’я об’єкта та імена властивостей **чутливі до регістру**.

Остання властивість у списку

Теорія: Об’єкт JavaScript

Остання властивість у списку може закінчуватися комою:

```
let user = {
  name: "Іван",
  age: 30,
}
```

Це називається “кінцевою” або “висячою” комою. Вона полегшує додавання/видалення/переміщення по властивостях, оскільки всі рядки стають однаковими.

Створення об’єкта

Теорія: Об’єкт JavaScript

Створення об’єкта починається з визначення та ініціалізації змінної. Наприклад, створення об’єкту за допомогою фігурних дужок:

```
const address = {};
```

Дуже поширеним є створення об’єкта за допомогою літералу об’єкта, з одночасним заповненням вмісту об’єкта, наприклад,

```
const address = {
      street: '13/25 New Avenue',
      city: 'New Heaven, ',
      country: 'Ukraine'
  },
```

Щоб створити ключ в об’єкті, **не потрібні ключові слова let або const**. Оскільки об’єкти є динамічними, можна додавати або змінювати властивості без оголошення змінної.

Конструктор Object()

Теорія: Об’єкт JavaScript

Об’єкти можна створювати також за допомогою **конструктора Object().**

Object.create() дозволяє точно налаштувати процес створення об’єкта.

За допомогою Object.create() можна створювати об’єкти з визначеним прототипом, а також деякими властивостями.

```
o = {};
// Is equivalent to:
o = Object.create(Object.prototype);

// Create a new object whose prototype is a new, empty
// object and add a single property 'p', with value 42.

o = Object.create({}, { p: { value: 42 } });
```

Екземпляр Object

Теорія: Об’єкт JavaScript

Майже всі об’єкти в JavaScript є екземплярами Object; типовий об'єкт успадковує властивості (включно з методами) від Object.prototype, хоча ці властивості можуть бути затінені (так само, як перевизначені). Єдині об’єкти, які не успадковують від Object.prototype, це об’єкти з нульовим прототипом або ті, що походять від інших об’єктів нульового прототипу.

Зміни в об’єкті Object.prototype бачать усі об’єкти через ланцюжок прототипів, якщо тільки властивості та методи, що підлягають цим змінам, не перевизначені далі в ланцюжку прототипів. Це забезпечує дуже потужний, але потенційно небезпечний механізм для перевизначення або розширення поведінки об’єкта. Щоб зробити його більш безпечним, Object.prototype є єдиним об’єктом у ядрі мови JavaScript, який має незмінний прототип - прототип Object.prototype завжди нульовий і не змінюється.

Доступ до властивостей об’єкта

Теорія: Об’єкт JavaScript

Отримати доступ до властивостей об’єкта можна за допомогою крапкової нотації.

Ім’я об’єкта (address) діє як простір імен - його потрібно ввести спочатку, щоб отримати доступ до будь-чого всередині об’єкта.

Далі ви пишете крапку, потім елемент, до якого ви хочете отримати доступ - це може бути назва простої властивості, елемент властивості масиву або виклик одного з методів об’єкта, наприклад:
address.street;
address.city;
Address.country;

За допомогою крапкової нотації ви можете отримати доступ до властивостей, які об’єкт не мав раніше. Наприклад, якщо об’єкт address не має властивості zip, доступ до неї поверне **«undefined»**.

Додати властивість до об’єкта

Теорія: Об’єкт JavaScript

Ви можете **додати** властивість zip до об’єкта address за допомогою крапкової нотації, а також можете **змінити** будь-яку властивість, призначивши їй нове значення.

```
address.zip = '001 001';
address.city =  'Kyiv';

console.log(
   address.street,
   address.city,
   address.country,
   address.zip
);
```

Нотація в дужка

Теорія: Об’єкт JavaScript

Нотація в дужках забезпечує альтернативний спосіб доступу до властивостей об’єкта. Замість використання крапкової нотації:

```
address.street,
address.city,
address.country,
```

Можна використовувати квадратні дужки:

```
address['street'],
address['city'],
address['country'],
```

Крапкова нотація, як правило, є кращою, ніж дужкова, оскільки вона є більш короткою та легшою для читання. Однак у деяких випадках потрібно використовувати квадратні дужки. Наприклад, якщо ім’я властивості містить пробіл

```
address['zip code'] = '001 002';
```

Значення властивості об’єкта

Теорія: Об’єкт JavaScript

Значення властивості об’єкта може бути будь-якого типу.

```
const product = {
    name: "Apple Watch",
    price: 425.99,
    stars: 5,
    category: 5,
};
```

Властивість об’єкта сама може бути об’єктом

Властивість об’єкта сама може бути **об’єктом**. Наприклад

```
const contacts = {
   address: {
       street: '13/25 New Avenue',
       city: 'Kyiv',
       zip: '001 001',
       country: 'Ukraine'
   },
};
```

Щоб отримати доступ до таких властивостей за допомогою крапкової нотації:

```
contacts.address.city;
```

Щоб отримати доступ до таких властивостей за допомогою нотації в дужках:

```
console.log(contacts['address']['city']);
```

Шаблон об’єкта

Теорія: Об’єкт JavaScript

Для створення об’єкта можна визначити шаблон об’єкта - набір методів і властивостей, які він може мати - а потім створити скільки завгодно об’єктів, просто оновлюючи значення властивостей, які відрізняються.

```
function createProduct(name, price, category, stars) {
    const obj = {};
    obj.name = name;
    obj.price = price;
    obj.category = category;
    obj.stars = stars;
    return obj;
}

const product1 = createProduct('Product1', 11.11, 3, 5);
const product2 = createProduct('Product2', 22.11, 1, 5);
```

Обчислювані властивості

Ми можемо використовувати квадратні дужки в літеральній нотації для створення обчислюваної властивості:

```
let fruit = prompt("Які фрукти купити?", "apple");

let bag = {
  [fruit]: 5, // назву властивості взято зі змінної fruit
};

alert( bag.apple ); // 5 якщо fruit="apple"
```

Значення обчислюваної властивості просте: [fruit] означає, що назву властивості слід брати з fruit. І якщо відвідувач введе слово "apple", то в об’єкті bag тепер буде лежати властивість {apple: 5}.

По суті, це працює так само як:

```
let fruit = prompt("Які фрукти купити?", "apple");
let bag = {};
// назву властивості взято зі змінної fruit
bag[fruit] = 5;
```

Перевірка існування властивості, оператор “in”

Теорія: Об’єкт JavaScript

Спроба прочитати відсутню властивість просто повертає undefined. Тому ми можемо легко перевірити, чи існує властивість:

```
let user = {};
alert( user.noSuchProperty === undefined ); // true означає "такої властивості немає"
```

Для цього також є спеціальний оператор "in". Синтаксис такий: "key" in object

Наприклад:

```
let user = { name: "Іван", age: 30 };
alert( "age" in user ); // true, user.age існує
alert( "blabla" in user ); // false, user.blabla не існує
```

Зліва від оператора in повинно бути ім’я властивості. Зазвичай це рядок в лапках.
Якщо ми опускаємо лапки, це означає, що ми вказуємо змінну, в якій знаходиться ім’я властивості:

```
let user = { age: 30 };
let key = "age";
alert( key in user ); // true, властивість "age" існує
```

Метод об’єкта

Теорія: Об’єкт JavaScript

Щоб створити метод в об’єкті, потрібні ключ і значення, де значення визначено за допомогою ключового слова **function**.

```
const contacts = {
    address: {
        street: '13/25 New Avenue',
        city: 'Kyiv',
        zip: '001 001',
        country: 'Ukraine'
    },
    info:function() {
        return "This is contact information";
    }
};

console.log(contacts.info());
```

Метод об’єкта (альтернативний синтаксис)

Теорія: Об’єкт JavaScript

Існує альтернативний синтаксис, за допомогою якого можна визначити метод більш ефективним способом, передбачає відкидання ключового слова "function" і двокрапки (:).

За допомогою короткого синтаксису методу властивість автоматично визначається як метод, що робить ключове слово "function" марним.

```
const contacts = {
    address: {
        street: '13/25 New Avenue',
        city: 'Kyiv',
        zip: '001 001',
        country: 'Ukraine'
    },
    info() {
        return "This is contact information";
    }
};
```

Видалити властивість об’єкта

Теорія: Об’єкт JavaScript

Видалити властивість в об’єкті дуже просто. JavaScript має спеціальне ключове слово delete, яке дозволяє видалити будь-які властивості.

```
const contacts = {
   address: {
       street: '13/25 New Avenue',
       city: 'Kyiv',
       zip: '001 001',
       country: 'Ukraine'
   },
};

delete contacts.zip;
```

Додавання властивості до об’єкта

Теорія: Об’єкт JavaScript

Одним з корисних аспектів нотації в квадратних дужках є те, що її можна використовувати для динамічного встановлення не лише значень властивостей, але й імен членів.

```
const call_center = {
   note: 'This number is toll free if calling from Ukraine.otherwise we advise you to use the',
   phone: '+38 555 444 333',
};

contacts[call_center] = call_center;
console.log(contacts);
```

Додавання властивості до об’єкта за допомогою такого методу неможливо з точковою нотацією, яка може приймати лише літеральне ім’я члена, а не значення змінної, що вказує на ім’я.

Метод Object.entries

Теорія: Об’єкт JavaScript

Метод **Object.entries()** повертає список пар властивостей ключ-значення з рядковими ключами.

Кожна пара ключ-значення - це масив із двох елементів: перший елемент - це ключ властивості (який **завжди є рядком**), а другий - значення властивості.

```
console.log(Object.entries(contacts));
```

Метод Object.keys

Теорія: Об’єкт JavaScript

Якщо вам потрібні лише ключі властивостей, використовуйте метод **Object.keys()**. \

```
console.log(Object.keys(contacts));
```

Метод Object.values

Теорія: Об’єкт JavaScript

Якщо вам потрібні лише значення властивостей, використовуйте натомість **Object.values().**

```
console.log(Object.values(contacts));
```

Метод Object.hasOwn

Теорія: Об’єкт JavaScript

Метод Object.hasOwn() повертає true, якщо вказаний об’єкт має вказану властивість як власну властивість. Якщо властивість успадковано або не існує, метод повертає false.

```
// Використання hasOwn для перевірки існування властивості
// як визначити, чи містить об’єкт прикладу властивість під назвою prop.

const example = {};
Object.hasOwn(example, "prop"); // false - 'prop' has not been defined

example.prop = "exists";
Object.hasOwn(example, "prop"); // true - 'prop' has been defined

example.prop = null;
Object.hasOwn(example, "prop"); // true - own property exists with value of null

example.prop = undefined;
Object.hasOwn(example, "prop"); // true - own property exists with value of undefined
```

Метод Object.hasOwnProperty

Теорія: Об’єкт JavaScript

Метод hasOwnProperty() екземплярів Object повертає логічне значення, яке вказує, чи має цей об’єкт вказану властивість як власну (на відміну від її успадкування).

```
console.log(Object.hasOwn(contacts, call_center));
```

Рекомендується замість Object.prototype.hasOwnProperty() використовувати метод Object.hasOwn, оскільки hasOwn працює для об’єктів із нульовим прототипом і з об’єктами, які перевизначили успадкований метод hasOwnProperty().
Хоча можна вирішити ці проблеми, викликавши Object.prototype.hasOwnProperty() для зовнішнього об’єкта, Object.hasOwn() більш інтуїтивно зрозумілий.

Метод Object.is

Теорія: Об’єкт JavaScript

Метод Object.is() визначає, чи є два значення однаковими.
Object.is(value1, value2)
Два значення є однаковими, якщо виконується одне з наступного:

- обидва невизначені
- обидва нульові
- обидва істинні або обидва хибні
- обидва рядки однакової довжини з однаковими символами в однаковому порядку
- обидва значення посилаються на той самий об’єкт у пам’яті
- обидва BigInt з однаковим числовим значенням
- обидва символи посилаються на одне значення символу
- обидва числа і обидва +0, або обидва -0, або обидва NaN
- або обидва ненульові, не NaN, і мають однакове значення

Object.is() не еквівалентний операторам == та ===. .

Ключове слово this

Теорія: Об’єкт JavaScript

Ключове слово this посилається на поточний об’єкт, усередині якого записується код. Ключове слово this - це особливе ключове слово в JavaScript, яке є найважливішим при використанні **всередині функції в об’єкті**.

```
address: {
      street: '13/25 New Avenue',
      city: 'Kyiv',
      zip: '001 001',
      country: 'Ukraine',
      our_address (){
         return `Our address: ${this.street},  ${this.city}, ${this.country}`;
      },
  },

console.log(contacts.address.our_address());
```

Оператор Spread

Теорія: Об’єкт JavaScript

Оператор **Spread** (поширення) об’єктів **вибирає всі пари ключ-значення** об’єкта та **копіює їх в новий об’єкт**.

```
const product = {
   name: "Apple Watch",
   price: 425,
};

console.log(product);
```

Оператор Spread визначається **трьома крапками перед об’єктом**, який потрібно поширити.

```
const productInCart ={...product, amount: 2};

console.log('Total in cart: ', productInCart.price * productInCart.amount);
```

Деструктуризація об’єктів

Теорія: Об’єкт JavaScript

Деструктуризація об’єктів дозволяє видобути значення з об’єкта та призначати їх окремим змінним. Щоб виконати деструктуризацію об’єкта, використовується шаблон деструктуризації в лівій частині оператора присвоєння, а об’єкт, з якого потрібно отримати значення, – у правій частині. Наприклад:

```
const person = { name: 'lawal', age: 39 };
const { person, age } = person;
console.log(name); // 'lawal'
console.log(age); // 39
```

Деструктуризація об’єкта також дає змогу вказати значення за замовчуванням на випадок, якщо властивість, яку ви хочете отримати, не існує в об’єкті. Ви також можете перейменовувати змінні, які вилучаються, за допомогою псевдоніма, що дає вам більше контролю над структурою та іменуванням видобутих значень.

Optional chaining ?

Теорія: Об’єкт JavaScript

Optional chaining ?. зупиняє обчислення, якщо **value**, що передує ?., має значення **undefined** або **null** і повертає **undefined**.

Іншими словами, **value?.prop** працює як value.prop, якщо value існує,

інакше (якщо value **undefined**/**null**) повертає **undefined**.

Безпечний спосіб отримати доступ до user.address.street за допомогою ?.:

```
let user = {}; // user has no address
alert( user?.address?.street ); // undefined (no error)
```

Код короткий і зрозумілий, дублювання взагалі відсутнє

Цикл "for..in"

Теорія: Об’єкт JavaScript

Для перебору всіх властивостей об’єкта використовується цикл for..in. Цей цикл відрізняється від вивченого раніше циклу for(;;).

```
for (key in object) {
 // тіло циклу виконується для кожної властивості об’єкта
}
```

Наприклад, виведемо всі властивості user:

```
let user = {
 name: "Іван",
 age: 30,
 isAdmin: true
};

for (let key in user) {
 // ключі
 alert( key );  // name, age, isAdmin
 // значення ключів
 alert( user[key] ); // Іван, 30, true
}
```

Об'єкти

Створення об’єкта calculator, що має механізм ланцюжкових викликів

Створення об'єкта calculator

Практика: Об’єкт JavaScript

Для створення об'єкта calculator із механізмом ланцюжкових викликів (method chaining) кожен метод об'єкта повинен повертати посилання на сам об'єкт (тобто this).

Властивість result: Зберігає поточний стан обчислень:

```
const calculator = {
 result: 0,
```

Повернення this

Практика: Об’єкт JavaScript

Повернення this: Методи read, add, subtract, multiply та divide модифікують значення this.result і наприкінці виконують return this.

Це дозволяє відразу викликати наступний метод через крапку.

Метод read

Практика: Об’єкт JavaScript

Повернення this:

```
read(value) {
   this.result = value;
   return this;
 },
```

Метод read модифікує значення this.result і наприкінці виконують return this.

Метод add

Практика: Об’єкт JavaScript

Повернення this:

```
add(value) {
   this.result += value;
   return this;
 },
```

Метод add модифікує значення this.result і наприкінці виконують return this.

Це дозволяє відразу викликати наступний метод через крапку.

Метод subtract

Повернення this:

```
subtract(value) {
   this.result -= value;
   return this;
 },
```

Метод subtract модифікує значення this.result і наприкінці виконують return this.

Це дозволяє відразу викликати наступний метод через крапку.

Метод multiply

Практика: Об’єкт JavaScript

Повернення this:

```
multiply(value) {
   this.result *= value;
   return this;
 },
```

Метод multiply модифікує значення this.result і наприкінці виконують return this.

Це дозволяє відразу викликати наступний метод через крапку.

Метод divide

Практика: Об’єкт JavaScript

Повернення this:

```
divide(value) {
  if (value === 0) {
    console.error("Помилка: Ділення на нуль!");
    return this;
  }
  this.result /= value;
  return this;
},
```

Метод divide модифікує значення this.result і наприкінці виконують return this.

Це дозволяє відразу викликати наступний метод через крапку.

Метод getResult

Повернення this:

```
getResult() {
  return this.result;
},
```

Метод getResult(): Термінальний метод.
Він повертає чисте числове значення, після чого ланцюжок переривається.

Метод reset

Практика: Об’єкт JavaScript

Повернення this:

```
  reset() {
    this.result = 0;
    return this;
  }
};
```

Використання calculator

Практика: Об’єкт JavaScript

```
// Приклад використання:

const finalResult = calculator
 .read(10)       // Початкове значення 10
 .add(5)        // 10 + 5 = 15
 .subtract(3)    // 15 - 3 = 12
 .multiply(2)    // 12 * 2 = 24
 .divide(4)      // 24 / 4 = 6
 .getResult();   // Отримання фінального результату

console.log(finalResult); // Виведе: 6
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua